

CYE 5153 LoS – 4_Lab

Srinivas P Bodke

261100690018

Part A – Basic grep and Filtering

Display all lines containing the word failed.

```
(sri@kali)-[~/Desktop]
$ ls
security_log.txt

(sri@kali)-[~/Desktop]
$ grep 'Failed' security_log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4
444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4
444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4
444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2
222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port
2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port
2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port
2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port
2222
```

Display all lines containing failed, ignoring case.

```
(sri@kali)-[~/Desktop]
$ grep -i 'Failed' security_log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4
444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4
444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4
444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2
222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port
2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port
2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port
2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port
2222
```

Display all lines that contain either Failed or Denied.

```
(sri@kali)-[~/Desktop]
$ egrep 'Failed|Denied' security_log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Display all lines that do not contain the word INFO.

```
(sri@kali)-[~/Desktop]
$ grep -v 'INFO' security_log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:13:02] WARNING Multiple authentication failures from 203.0.113.45
[2025-08-18 08:15:44] ERROR Connection denied from 198.51.100.77 port 9001
[2025-08-18 08:16:05] Suspicious connection from 198.51.100.77 to evilc2.ddns.net:9001
[2025-08-18 08:16:19] Suspicious outbound request https://evilc2.ddns.net/update.php
[2025-08-18 08:18:07] WARNING Request to http://malware-test.example.org/download.php from 10.20.30.40
[2025-08-18 08:18:20] Suspicious file download evil.exe from http://malware-test.example.org/evil.exe
[2025-08-18 08:18:32] ERROR file evil.exe hash MD5=44d88612fea8a8f36de82e1278abb02f
[2025-08-18 08:19:45] Suspicious file invoice_2025.pdf downloaded from https://phishing-example.com/invoice_2025.pdf
[2025-08-18 08:20:10] WARNING Email received from attacker123@phishing-example.com
[2025-08-18 08:20:51] Suspicious email phishing.alert@evilmail.ru contained malicious attachment
[2025-08-18 08:22:03] ERROR Exploit attempt CVE-2025-12345 from 203.0.113.99
[2025-08-18 08:22:21] WARNING Vulnerability CVE-2024-3094 detected on server
[2025-08-18 08:22:48] Suspicious exploit reference CVE-2025-987654 targeting https://target.example.com/login.php
[2025-08-18 08:24:01] Suspicious SHA1 hash=3395856ce81f2b7382dee72602f798b642f14140 file updater.sh
[2025-08-18 08:24:25] Suspicious SHA256 hash=275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f file payload.exe
[2025-08-18 08:25:18] WARNING file payload.exe stored at C:\Users\Public\Downloads\payload.exe
[2025-08-18 08:26:05] Suspicious script stored at /tmp/updater.sh
[2025-08-18 08:26:31] WARNING Configuration modified /etc/ssh/sshd_config
[2025-08-18 08:27:02] Suspicious persistence entry added to /etc/cron.d/system_update
```

Display all lines containing the exact word root.

```
(sri@kali)-[~/Desktop]
$ grep -w 'root' security_log.txt
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:27:24] Suspicious crontab created for user root
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
```

Display all occurrences of the word attack along with their line numbers.

```
(sri@kali)-[~/Desktop]
$ grep -ni 'attack' security_log.txt
17:[2025-08-18 08:20:10] WARNING Email received from attacker123@phishing-example.com
```

Count the number of lines containing ERROR.

```
(sri@kali)-[~/Desktop]
$ grep -c 'ERROR' security_log.txt
10
```

Display only the filenames of files whose contents contain the word malware, assuming multiple log files are available.

```
(sri@kali)-[~/Desktop]
$ grep -l 'malware' *.txt
security_log.txt
```

Display the first five lines containing Suspicious.

```
(sri@kali)-[~/Desktop]
$ grep 'Suspicious' security_log.txt | head -n 5
[2025-08-18 08:16:05] Suspicious connection from 198.51.100.77 to evilc2.ddns.net:9001
[2025-08-18 08:16:19] Suspicious outbound request https://evilc2.ddns.net/update.php
[2025-08-18 08:18:20] Suspicious file download evil.exe from http://malware-test.example.org/evil.exe
[2025-08-18 08:19:45] Suspicious file invoice_2025.pdf downloaded from https://phishing-example.com/invoice_2025.pdf
[2025-08-18 08:20:51] Suspicious email phishing.alert@evilmail.ru contained malicious attachment
```

Display the last five failed-login entries.

```
(sri@kali)-[~/Desktop]
$ grep -Ei 'failed.*login|login.*failed' security_log.txt | tail -5
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Part B – Regular Expression Extraction

Extract all IPv4 addresses from the log.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt
```

Extract all IP addresses belonging to the 10.x.x.x private network.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt
```

Extract all addresses belonging to the 192.168.x.x network.

```
grep -Eo '\b192\.- 168(\- [0-9](1,3))\{2\}\b' security_log.txt
```

Extract all email addresses.

```
grep -Eo '[[[:alnum:]]._%+-]+@[[[:alnum:]].-]+\.[[:alpha:]]{2,}' security_log.txt
```

Extract only email addresses whose domain ends in .com.

```
grep -Eo '[[[:alnum:]]._%+-]+@[[[:alnum:]].-]+\com\b' security_log.txt
```

Extract email addresses whose username contains at least one digit.

```
grep -Eo '[[[:alnum:]]._%+-]*[8-9] [[[:alnum:]]._8+-]*@[[[:alnum:]].-]+\ [[[:alpha:]](2,)' security_log.txt
```

Extract all URLs beginning with http:// or https://.

```
egrep -o 'https?://[^\[:space:]]+' security_log.txt
```

Extract only HTTPS URLs.

```
grep -Eo 'https://[^\[:space:]]+' security_log.txt
```

Extract URLs ending with .php.

```
grep -Eo 'https?://[^\s:]+\-php\b' log.txt
```

Extract all CVE identifiers in the form: CVE-YYYY-NNNN

```
grep -Eo 'CVE-[0-9]{4}-[0-9]{4}' security_log.txt
```

Extract only CVEs reported for the year 2025.

```
grep -Eo 'CVE-2025-[0-9]{4}' security_log.txt
```

Extract all 32-character hexadecimal MD5 hashes.

```
grep -Eio '\b[0-9a-f]{32}\b' security_log.txt
```

Extract all 40-character hexadecimal SHA-1 hashes.

```
grep -Eio '\b[0-9a-f]{40}\b' security_log.txt
```

Extract all 64-character hexadecimal SHA-256 hashes.

```
grep -Eio '\b[0-9a-f]{64}\b' security_log.txt
```

Extract all timestamps having the format:

YYYY-MM-DD HH:MM:SS

```
egrep '\b[0-9]{4}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}\b' security_log.txt
```

Part C – Anchors and Regex Patterns

Display all lines beginning with a timestamp.

```
grep -E '^ [0-9]{4}-[0-9]{2}-[0-9]{2}' security_log.txt
```

Display all lines beginning with the word Suspicious.

```
grep '^Suspicious' security_log.txt
```

Display lines ending with a numeric value.

```
grep -E '[0-9]+$' security_log.txt
```

Display lines beginning with ERROR or WARNING.

```
grep -E '^(ERROR|WARNING)' security_log.txt
```

Display lines whose first character is a digit.

```
grep -E '[0-9]' security_log.txt
```

Find lines ending in

.exe. `grep -E '\.exe$'`
`security_log.txt`

Find lines containing filenames ending in .exe, .sh, or .pdf.

`grep -E '\.(exe|sh|pdf)' security_log.txt`

Identify lines containing the exact word root, but not words such as rooted.

`grep -E '\broot\b' security_log.txt`

Display blank lines in the file.

`grep -E '^$' security_log.txt`

Display all nonblank lines.

`grep -E '.*' security_log.txt`

Part D – Linux Filters and Pipelines

Extract all IPv4 addresses, sort them, and remove

duplicates. `grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b'`
`security_log.txt | sort -u`

Count the number of unique IPv4 addresses present in the log.

`grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt | sort -u | wc -l`

Extract all IP addresses and display them in sorted order.

`Grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt | sort`

Determine which IP addresses appear more than

once. `grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b'`
`security_log.txt | sort`

Determine the IP address that occurs most frequently.

`grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt | sort | uniq -c | sort -nr | head -1`

Extract all usernames from authentication entries and display unique usernames.

`grep -Ei 'authentication|login' security_log.txt | grep -Eo 'username[=:]^[^ ]+' | cut -d'=' -f2 | sort -u`

Count the number of authentication attempts associated with each username. `grep -Ei`

`'authentication|login' security_log.txt | grep -Eo 'username[=:]^[^ ]+' | cut -d'=' -f2 | sort | uniq -c | sort -nr`

Extract all HTTP status codes and determine how many times each status code occurs. `grep -Eo '\b[1-5][0-9]{2}\b' security_log.txt | sort | uniq -c | sort -nr`

Display the five most frequently occurring IP addresses.

`grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt | sort | uniq -c | sort -nr | head -5`

Display the three most frequently occurring usernames. `grep -Ei 'authentication|login' security_log.txt | grep -Eo 'username[=:]\[^\]+' | cut -d '=' -f2 | sort | uniq -c | sort -nr | head -3`

Extract all destination ports and display them in numerical order.

`grep -Eo 'port[=:]\[0-9\]+' security_log.txt | grep -Eo '\[0-9\]+' | sort -n`

Remove duplicate port numbers.

`grep -Eo 'port[=:]\[0-9\]+' security_log.txt | grep -Eo '\[0-9\]+' | sort -n -u`

Count the number of unique destination ports.

`grep -Eo 'port[=:]\[0-9\]+' security_log.txt | grep -Eo '\[0-9\]+' | sort -n -u | wc -l`

Extract all file extensions appearing in the log and determine their frequency. `grep -Eo '\.[A-Za-z0-9]+([[:space:]]|$)' security_log.txt | tr -d ' ' | sort | uniq -c | sort -nr`

Find the five most frequently occurring domains.

`grep -Eo '\b([A-Za-z0-9-]+\.[A-Za-z]{2,})\b' security_log.txt | sort | uniq -c | sort -nr | head -5`

Part E – Combining grep Conditions

Display lines containing both an IP address and the word Failed.

`grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | grep 'Failed'`

Display lines containing both root and Failed.

`grep 'root' security_log.txt | grep 'Failed'`

Display lines containing either download or upload.

`grep -Ei 'download|upload' security_log.txt`

Display suspicious entries containing either .exe or .sh.

`grep -i 'Suspicious' security_log.txt | grep -Ei '\.exe|.sh'`

Display lines containing a CVE identifier and an IP address.

`grep -E 'CVE-[0-9]{4}-[0-9]{4,}' security_log.txt | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}'`

Display lines containing both a filename and a hash value.

```
grep -Ei '\.[A-Za-z0-9]+.*[0-9a-fA-F]{32,}' security_log.txt
```

Display entries containing an IP address followed by a port.

```
grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}:[0-9]+' security_log.txt
```

Display HTTP requests containing both a URL and status code

```
404. grep -E 'https?:/' security_log.txt | grep -E '\b404\b'
```

Display authentication failures but exclude entries generated from 127.0.0.1.

```
grep -Ei 'authentication.*fail|failed.*login' security_log.txt | grep -v '127\.\0\.\0\.'1'
```

Display attack-related entries but exclude informational (INFO) messages.

```
grep -Ei 'attack' security_log.txt | grep -v 'INFO'
```

Part F – Negative Matching

Extract all IPv4 addresses except those beginning with

```
127. grep -Eo '\b(?!127\.)([0-9]{1,3}\.){3}[0-9]{1,3}\b'  
security_log.txt
```

Display all log entries except successful login events.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt | grep -v '^127\.'
```

Display all entries that do not contain INFO, DEBUG, or SUCCESS.

```
grep -Eiv 'successful login|login successful' security_log.txt
```

Find HTTP requests whose status code is not 200.

```
grep -E 'https?:/' security_log.txt | grep -Ev '\b200\b'
```

Display suspicious traffic excluding connections to 192.168.x.x.

```
grep -i 'Suspicious' security_log.txt | grep -Ev '192\.\168\.'
```

Extract email addresses except those belonging to the organization's

```
domain. grep -Eo '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}'  
security_log.txt | grep -v '@example\com$'
```

Part G – Redirection

Extract all IPv4 addresses and save them into: ips.txt

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b'  
security_log.txt > ips.txt
```

Extract all email addresses and save them into: emails.txt `grep -Eo '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' security_log.txt > emails.txt`

Extract all URLs and save them into:

urls.txt `grep -Eo 'https?:/[^\[:space:]]+' security_log.txt > urls.txt`

Extract all hashes and save them into: hashes.txt `grep -Eo '\b[0-9a-fA-F]{32}\b|\b[0-9a-fA-F]{40}\b|\b[0-9a-fA-F]{64}\b' security_log.txt > hashes.txt`

Extract failed-login entries and save them into: failed_login.txt `grep -Ei 'failed.*login|login.*failed|authentication.*failed' security_log.txt > failed_login.txt`

Extract suspicious entries and append them to:

incident_report.txt `grep -i 'Suspicious' security_log.txt >> incident_report.txt`

Store unique IP addresses in: unique_ips.txt `grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt | sort -u > unique_ips.txt`

Store IP-frequency statistics in: ip_frequency.txt `grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log.txt | sort | uniq -c | sort -nr > ip_frequency.txt`

Redirect normal output to results.txt and errors to

errors.txt. `grep -E 'ERROR|WARNING' security_log.txt > results.txt 2> errors.txt` **Extract all domains from the log entries containing Domain accessed, convert the domain**

names to lowercase, sort them alphabetically, remove duplicates, and save the result in domains.txt.

`grep 'Domain accessed' security_log.txt | grep -Eo '([A-Za-z0-9.-]+\.[A-Za-z]{2,})' | tr '[:upper:]' '[:lower:]' | sort -u > domains.txt`

identify all IP addresses involved in failed-login attempts. Convert the output into a clean list containing one unique IP per line and save it as failed_login_ips.txt.

`grep -Ei 'failed.*login|login.*failed|authentication.*failed' security_log.txt | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort -u > failed_login_ips.txt`

Combining filters

Extract all failed-login usernames and save them into: failed_users.txt `grep -Ei`

`'failed.*login|login.*failed|authentication.*failed' security_log.txt | grep -Eo 'username[=][^\+]' | cut -d'=' -f2 | sort -u > failed_users.txt`

Extract all domains and save them in lowercase into: domains.txt
grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' security_log.txt | tr '[:upper:]' '[:lower:]' > domains.txt

Extract all MD5 hashes, convert them to uppercase, and save them into: md5_hashes.txt
grep -Eo '\b[0-9a-fA-F]{32}\b' security_log.txt | tr '[:lower:]' '[:upper:]' > md5_hashes.txt

Extract source IP addresses from failed-login records, sort them, remove duplicates, and redirect the result to: failed_ips.txt
grep -Ei 'failed.*login|login.*failed|authentication.*failed' security_log.txt | grep -Eo '\b([09]{1,3}\.){3}[0-9]{1,3}\b' | sort -u > failed_ips.txt

Create a file named clean_log.txt in which all square brackets are removed from the original log.
sed 's/[//g; s/]//g' security_log.txt > clean_log.txt

Create normalized_log.txt by converting the complete log file to lowercase.
tr '[:upper:]' '[:lower:]' < security_log.txt > normalized_log.txt

Extract all domain names, convert them to lowercase, sort them, remove duplicates, and save them to unique_domains.txt.
grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' security_log.txt | tr '[:upper:]' '[:lower:]' | sort -u > unique_domains.txt

Find all failed-login entries and extract only the usernames. grep -Ei 'failed.*login|login.*failed|authentication.*failed' security_log.txt | grep -Eo 'username[=:]([+]| cut -d'= ' -f2

Find all failed-login entries, extract only the source IP addresses, and display them in sorted order.

grep -Ei 'failed.*login|login.*failed|authentication.*failed' security_log.txt | grep -Eo '\b([09]{1,3}\.){3}[0-9]{1,3}\b' | sort

Extract IP addresses associated with failed logins and determine the unique IPs. grep -Ei 'failed.*login|login.*failed|authentication.*failed' security_log.txt | grep -Eo '\b([09]{1,3}\.){3}[0-9]{1,3}\b' | sort -u

Extract usernames involved in failed logins, convert them to uppercase, and remove duplicate values.

grep -Ei 'failed.*login|login.*failed|authentication.*failed' security_log.txt | grep -Eo 'username[=:]([+]| cut -d'= ' -f2 | tr '[:lower:]' '[:upper:]' | sort -u

Find all lines containing Domain accessed, extract only the domain names, and convert them to lowercase.

```
grep 'Domain accessed' security_log.txt | grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}' | tr '[:upper:]' '[:lower:]'
```

Find all lines containing MD5=, extract only the MD5 hash values, and convert hexadecimal letters to uppercase.

```
grep 'MD5=' security_log.txt | grep -Eo 'MD5=[0-9a-fA-F]{32}' | cut -d '=' -f2 | tr '[:lower:]' '[:upper:]'
```

Find entries containing User-Agent:, extract the user-agent field, and convert it to lowercase.

```
grep 'User-Agent:' security_log.txt | sed 's/.*User-Agent:[[:space:]]*//' | tr '[:upper:]' '[:lower:]'
```

Find all CVE-related lines, extract the CVE identifiers, and convert them to lowercase.

```
grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' security_log.txt | tr '[:upper:]' '[:lower:]'
```

Find all entries containing .exe, extract the relevant filename, and convert it to uppercase.

```
grep -Eo '[A-Za-z0-9_./-]+\.exe' security_log.txt | tr '[:lower:]' '[:upper:]'
```

Extract all domains from Domain accessed entries, convert them to uppercase, sort them, and remove duplicates.

```
grep 'Domain accessed' security_log.txt | grep -Eo '([A-Za-z0-9-]+\.)+[A-Za-z]{2,}'
```